Vehicle Replication

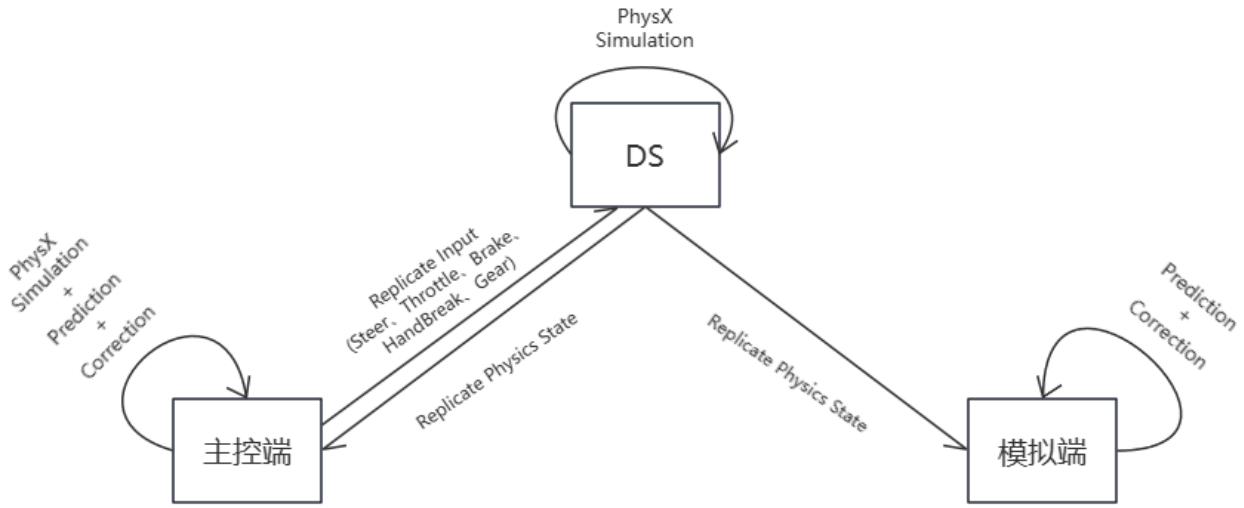
一、前言

载具的网络同步和角色（CMC）、Projectile 以及普通 Actor 移动同步有比较多的区别，主要在于载具需要依靠物理引擎（PhysX/Chaos）来进行物理模拟，物理模拟的介入，增加了如何保证三端网络同步的复杂度。

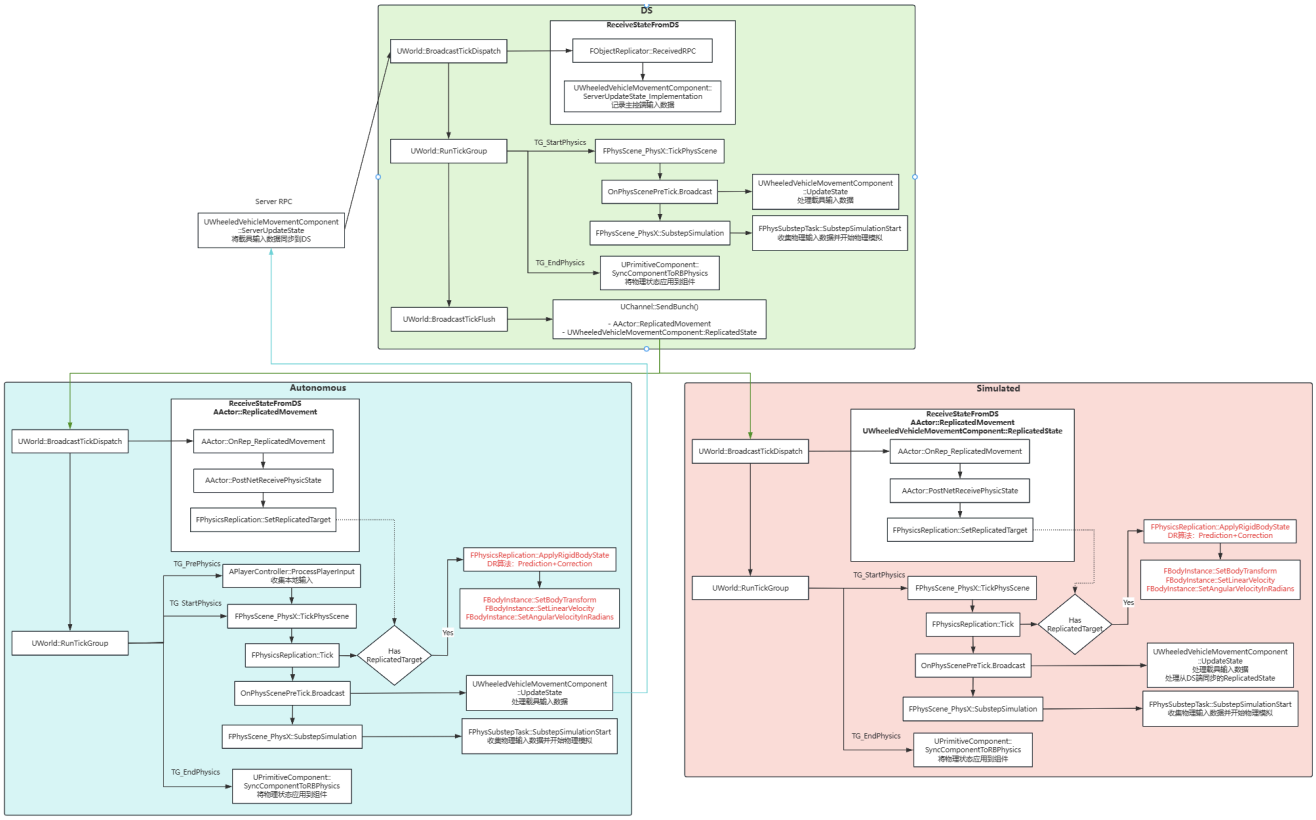
下面以 PhysX 物理引擎和 UE4 默认的物理同步方案为例，剖析物理同步原理和引擎底层源码

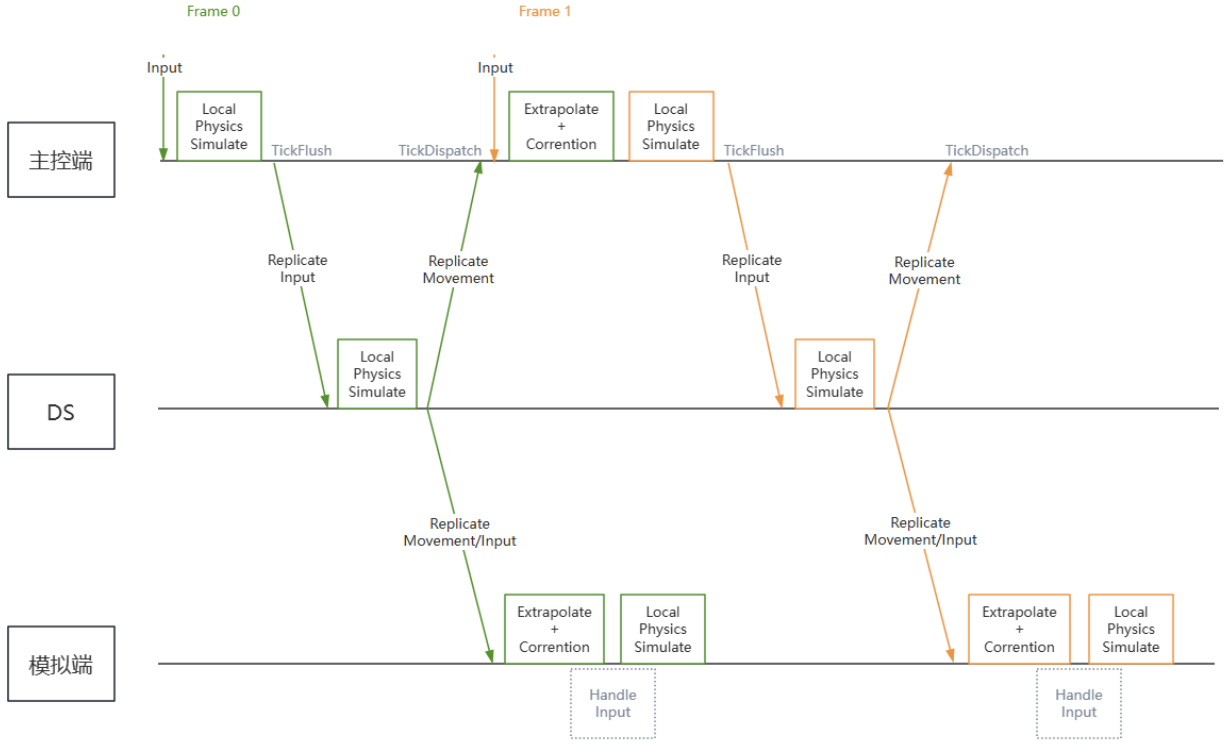
二、载具物理同步基本原理

2.1 基本原理



2.2 物理同步流程


DS
ReceiveStateFromDS
UWorld::BroadcastTickDispatch
FObjectReplicator::ReceivedRPC
UWheeledVehicleMovementComponent::
ServerUpdateState_Implementation
记录主控端输入数据
UWorld::RunTickGroup
TG_StartPhysics
FPhysScene_PhysX::TickPhysScene
OnPhysScenePreTick.Broadcast
UWheeledVehicleMovementComponent
::UpdateState
处理载具输入数据
FPhysScene_PhysX::SubstepSimulation
FPhysSubstepTask::SubstepSimulationStart
收集物理输入数据并开始物理模拟
TG_EndPhysics
UPrimitiveComponent::
SyncComponentToRBPhysics
将物理状态应用到组件
UWorld::BroadcastTickFlush
UChannel::SendBunch()
- AActor::ReplicatedMovement
- UWheeledVehicleMovementComponent::ReplicatedState
Server RPC
UWheeledVehicleMovementComponent
::ServerUpdateState
将载具输入数据同步到DS
Autonomous
ReceiveStateFromDS
AActor::ReplicatedMovement
UWorld::BroadcastTickDispatch
AActor::OnRep_ReplicatedMovement
AActor::PostNetReceivePhysicState
FPhysicsReplication::SetReplicatedTarget
TG_PrePhysics
APlayerController::ProcessPlayerInput
收集本地输入
TG_StartPhysics
FPhysScene_PhysX::TickPhysScene
UWorld::RunTickGroup
FPhysicsReplication::Tick
Has
ReplicatedTarget
Yes
FPhysicsReplication::ApplyRigidBodyState
DR算法：Prediction+Correction
FBodyInstance::SetBodyTransform
FBodyInstance::SetLinearVelocity
FBodyInstance::SetAngularVelocityInRadians
OnPhysScenePreTick.Broadcast
UWheeledVehicleMovementComponent
::UpdateState
处理载具输入数据
FPhysScene_PhysX::SubstepSimulation
FPhysSubstepTask::SubstepSimulationStart
收集物理输入数据并开始物理模拟
TG_EndPhysics
UPrimitiveComponent::
SyncComponentToRBPhysics
将物理状态应用到组件
Simulated
ReceiveStateFromDS
AActor::ReplicatedMovement
UWheeledVehicleMovementComponent::ReplicatedState
UWorld::BroadcastTickDispatch
AActor::OnRep_ReplicatedMovement
AActor::PostNetReceivePhysicState
FPhysicsReplication::SetReplicatedTarget
FPhysicsReplication::ApplyRigidBodyState
DR算法：Prediction+Correction
FBodyInstance::SetBodyTransform
FBodyInstance::SetLinearVelocity
FBodyInstance::SetAngularVelocityInRadians
UWorld::RunTickGroup
TG_StartPhysics
FPhysScene_PhysX::TickPhysScene
FPhysicsReplication::Tick
Has
ReplicatedTarget
Yes
OnPhysScenePreTick.Broadcast
UWheeledVehicleMovementComponent
::UpdateState
处理载具输入数据
处理从DS接收的ReplicatedState
FPhysScene_PhysX::SubstepSimulation
FPhysSubstepTask::SubstepSimulationStart
收集物理输入数据并开始物理模拟
TG_EndPhysics
UPrimitiveComponent::
SyncComponentToRBPhysics
将物理状态应用到组件


Frame 0
Frame 1
主控端
DS
模拟端
Input
Local
Physics
Simulate
TickFlush
TickDispatch
Input
Extrapolate
+
Corrention
Local
Physics
Simulate
TickFlush
TickDispatch
Replicate
Input
Replicate
Movement
Replicate
Input
Replicate
Movement
Local
Physics
Simulate
Local
Physics
Simulate
Replicate
Movement/Input
Replicate
Movement/Input
Extrapolate
+
Corrention
Local
Physics
Simulate
Extrapolate
+
Corrention
Local
Physics
Simulate
Handle
Input
Handle
Input

where we were.

The technique, *projective velocity blending*, works like this:

$$V_b = V_0 + (V_0' - V_0)\hat{T} \qquad \text{(Velocity blending)}$$

$$P_t = P_0 + V_b T_t + \frac{1}{2} A_0' T_t^2 \qquad \text{(Projecting from where we were)}$$

$$P_t' = P_0' + V_0' T_t + \frac{1}{2} A_0' T_t^2 \qquad \text{(Projecting from last known)}$$

$$Q_t = P_t + (P_t' - P_t)\hat{T}. \qquad \text{(Combined)}$$

And the red lines in Figure 1.3 show what it looks like in action.

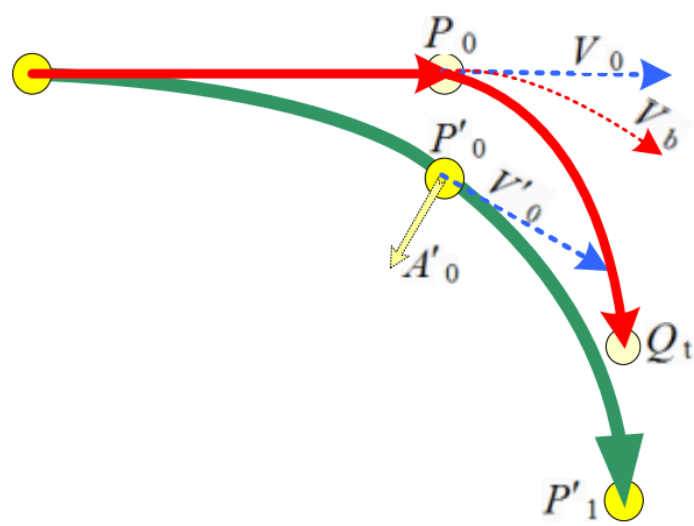


Figure 1.3. Dead reckoning with projective velocity blending (shown in red)

If you also have angular acceleration, just add it to `rotationAngle`. Next, compute the two projections and blend using a spherical linear interpolation. Use the last known angular velocity in both projections, just like the last known acceleration was used for both equations in the projective velocity blending technique:

$$R'_{\Delta t} = \text{quat}\left(R'_{\text{mag}} T_t, R'_{\text{dir}}\right) \qquad \text{(Impact of angular velocity)}$$

$$R_t = R'_{\Delta t} R_0 \qquad \text{(Rotated from where we were)}$$

$$R'_t = R'_{\Delta t} R'_0 \qquad \text{(Rotated from last known)}$$

$$R_t = \text{slerp}\left(\hat{T}, R_t, R'_t\right). \qquad \text{(Combined)}$$

This holds true for $\hat{T} < 1.0$. Once again, $\hat{T}$ is clamped at 1.0, so the math simplifies when $\hat{T} >= 1.0$:

$$R_t = R'_{\Delta t} R'_0. \qquad \text{(Rotated from last known)}$$

三、引擎底层代码剖析

四、技术方案扩展

五、总结